

Sistemas Operativos

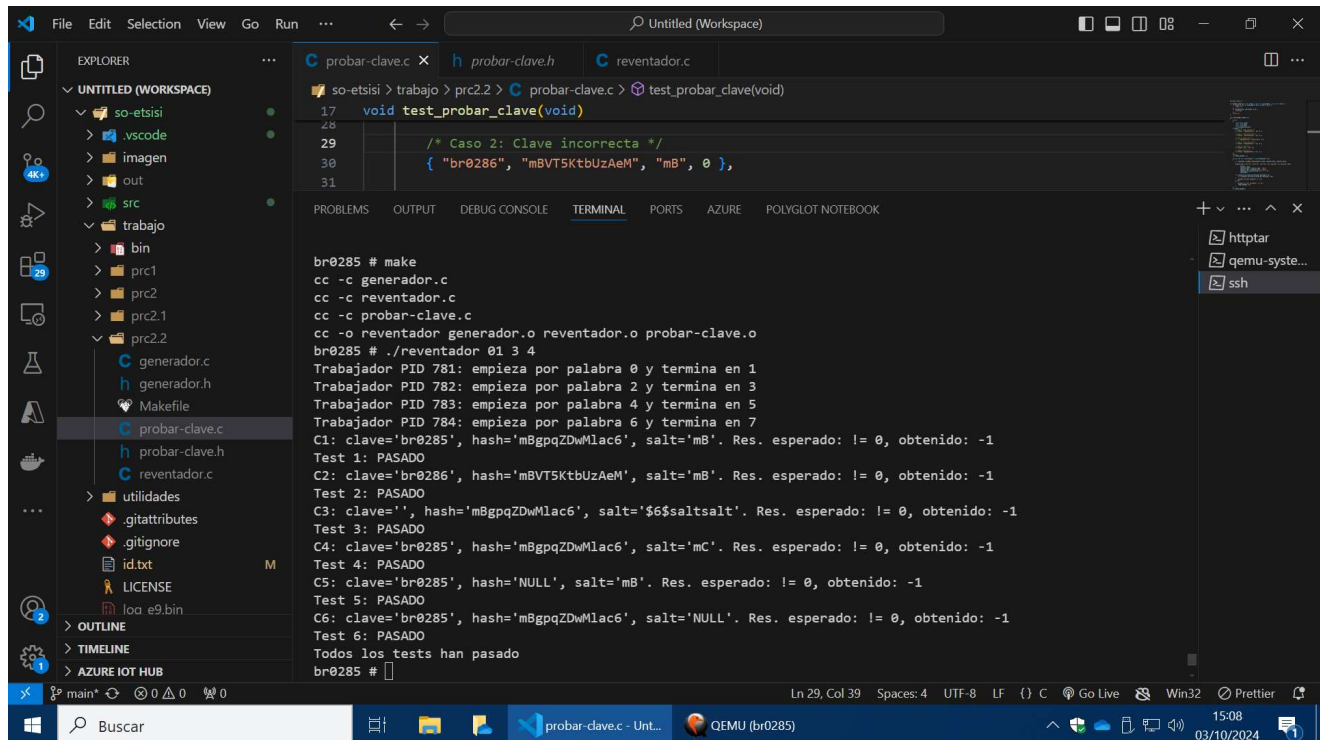
Práctica 2.2

Fecha: 3/10/2024-14/10/2024

Alumno: Yzan Martinez Manzano

Correo: yzan.martinez.manzano@alumnos.upm.es

1. Probar-clave



The screenshot shows the Visual Studio Code editor with a workspace named 'Untitled (Workspace)'. The Explorer panel on the left shows a project structure with folders like 'so-etsisi', 'trabajo', and 'prc2.2'. The file 'probar-clave.c' is open in the editor. The code defines a function 'test_probar_clave(void)' that runs a series of tests. The terminal at the bottom shows the output of these tests, including the compilation of 'probar-clave.c' and 'reventador.c', and the execution of 'reventador' with various test cases. The tests are labeled C1 through C6, each with a specific key, salt, and expected result. The output shows that all tests passed ('PASADO').

```
void test_probar_clave(void)
{
    /* Caso 2: Clave incorrecta */
    { "br0286", "mBVT5KtbUzAeM", "mB", 0 },
    ...
}
```

```
br0285 # make
cc -c generador.c
cc -c reventador.c
cc -c probar-clave.c
cc -o reventador generador.o reventador.o probar-clave.o
br0285 # ./reventador 01 3 4
Trabajador PID 781: empieza por palabra 0 y termina en 1
Trabajador PID 782: empieza por palabra 2 y termina en 3
Trabajador PID 783: empieza por palabra 4 y termina en 5
Trabajador PID 784: empieza por palabra 6 y termina en 7
C1: clave='br0285', hash='mBgpqZDwMlac6', salt='mB'. Res. esperado: != 0, obtenido: -1
Test 1: PASADO
C2: clave='br0286', hash='mBVT5KtbUzAeM', salt='mB'. Res. esperado: != 0, obtenido: -1
Test 2: PASADO
C3: clave='', hash='mBgpqZDwMlac6', salt='$6$saltsalt'. Res. esperado: != 0, obtenido: -1
Test 3: PASADO
C4: clave='br0285', hash='mBgpqZDwMlac6', salt='mC'. Res. esperado: != 0, obtenido: -1
Test 4: PASADO
C5: clave='br0285', hash='NULL', salt='mB'. Res. esperado: != 0, obtenido: -1
Test 5: PASADO
C6: clave='br0285', hash='mBgpqZDwMlac6', salt='NULL'. Res. esperado: != 0, obtenido: -1
Test 6: PASADO
Todos los tests han pasado
br0285 #
```

```
#define _MINIX
#include "probar-clave.h"
#include <unistd.h>
#include <stdio.h>
#include <string.h>
#include <errno.h>
int probar_clave(const char clave[], const char salt_hash[], const char salt[])
{
    const char *resultado = crypt(clave, salt);
    if (clave == NULL || salt_hash == NULL || salt == NULL)
    {
        return -1;
    }
    if (resultado == NULL)
    {
        return -1;
    }
    return strcmp(resultado, salt_hash) == 0 ? 0 : -1;
}
```

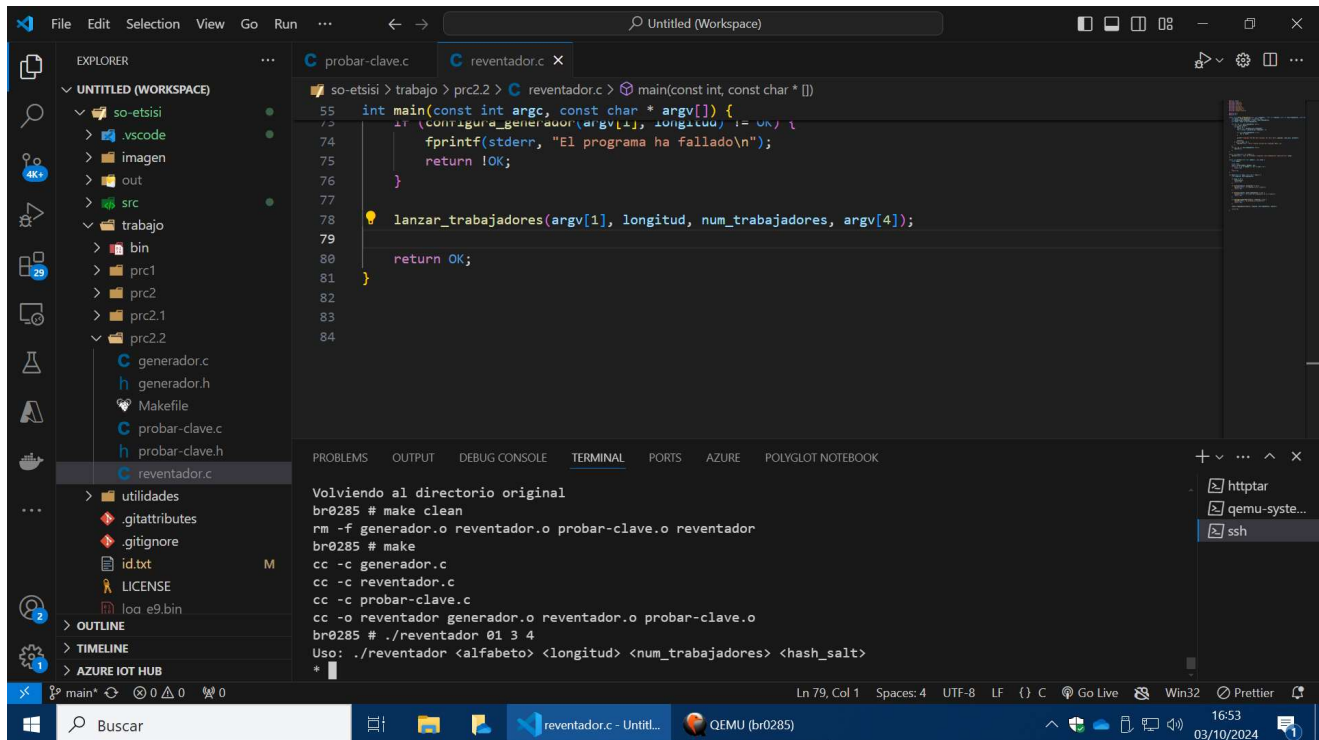
```

void test_probar_clave(void)
{
    struct
    {
        const char *clave;
        const char *hash;
        const char *salt;
        int es_clave_correcta;
    } casos[] = {
        /* Caso 1: Clave correcta */
        {"pw2003", "mBgpqZDwMlac6", "mB", 1},
        /* Caso 2: Clave incorrecta */
        {"bt0072", "mBVT5KtbUzAeM", "mB", 0},
        /* Caso 3: Clave vacía */
        {"", "mBgpqZDwMlac6", "$6$saltsalt", 0},
        /* Caso 4: Salt incorrecto */
        {"pw2003", "mBgpqZDwMlac6", "mC", 0},
        /* Caso 5: Hash nulo */
        {"pw2003", NULL, "mB", 0},
        /* Caso 6: Salt nulo */
        {"pw2003", "mBgpqZDwMlac6", NULL, 0}};

    int i;
    int todos_pasados = 1;
    for (i = 0; i < sizeof(casos) / sizeof(casos[0]); i++)
    {
        int resultado = probar_clave(casos[i].clave, casos[i].hash, casos[i].salt);
        printf("C %d: clave='%s', hash='%s', salt='%s'. Res. esperado: %s, obtenido:
%d\n",
            i + 1,
            casos[i].clave,
            casos[i].hash ? casos[i].hash : "NULL",
            casos[i].salt ? casos[i].salt : "NULL",
            casos[i].es_clave_correcta ? "0" : "!= 0",
            resultado);
        if ((casos[i].es_clave_correcta && resultado == 0) || (!casos[i].es_clave_correcta
&& resultado != 0))
        {
            printf("Test %d: Pasado\n", i + 1);
        }
        else
        {
            printf("Test %d: Fallado\n", i + 1);
            todos_pasados = 0;
        }
    }
    if (todos_pasados)
    {
        printf("Todos los tests han pasado\n");
    }
    else
    {
        printf("Algunos tests han fallado\n");
    }
}

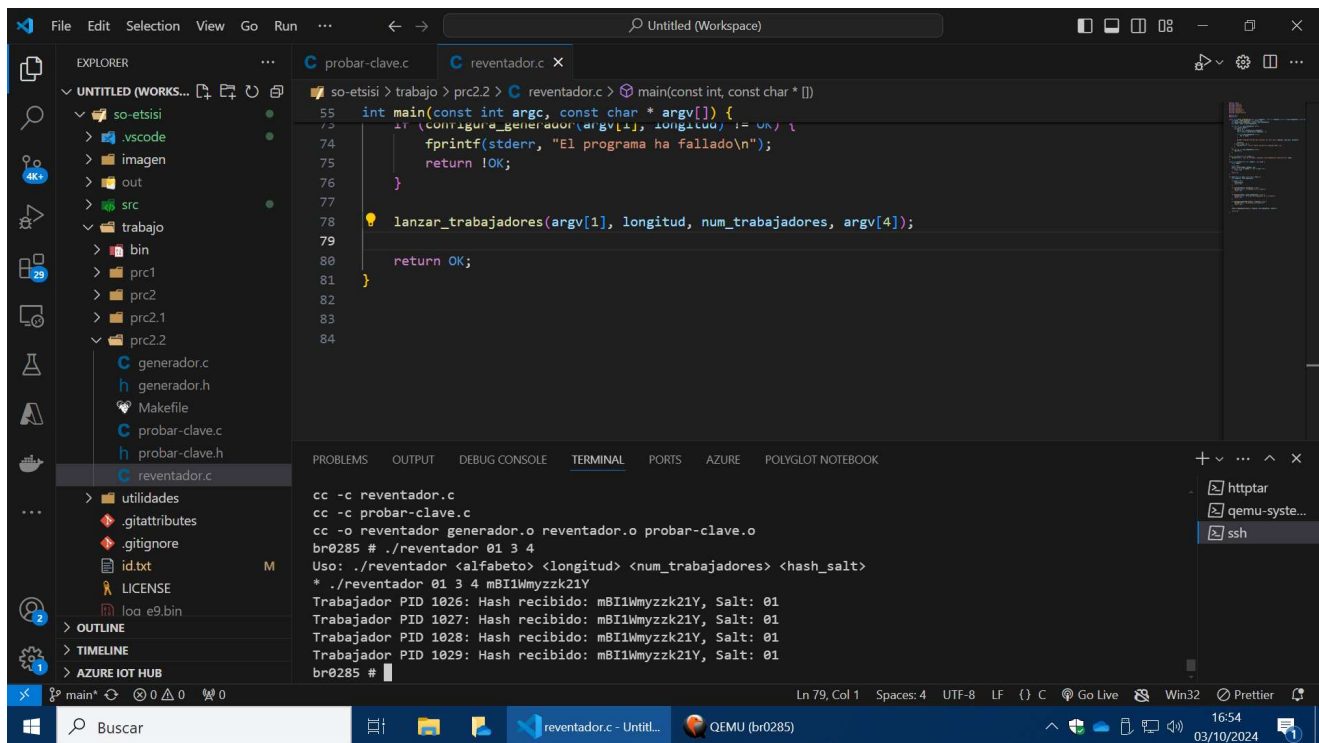
```

2. Nuevo argumento



```
so-etsisi > trabajo > prc2.2 > C reventador.c > main(const int, const char * [])
55 int main(const int argc, const char * argv[]) {
73     if (configura_generador(argv[1], longitud) != OK) {
74         fprintf(stderr, "El programa ha fallado\n");
75         return OK;
76     }
77
78     lanzar_trabajadores(argv[1], longitud, num_trabajadores, argv[4]);
79
80     return OK;
81 }
82
83
84
```

```
Volviendo al directorio original
br0285 # make clean
rm -f generator.o reventador.o probar-clave.o reventador
br0285 # make
cc -c generator.c
cc -c reventador.c
cc -c probar-clave.c
cc -o reventador generator.o reventador.o probar-clave.o
br0285 # ./reventador 01 3 4
Usa: ./reventador <alfabeto> <longitud> <num_trabajadores> <hash_salt>
*
```



```
cc -c reventador.c
cc -c probar-clave.c
cc -o reventador generator.o reventador.o probar-clave.o
br0285 # ./reventador 01 3 4
Usa: ./reventador <alfabeto> <longitud> <num_trabajadores> <hash_salt>
* ./reventador 01 3 4 mBI1Wmyzzk21Y
Trabajador PID 1026: Hash recibido: mBI1Wmyzzk21Y, Salt: 01
Trabajador PID 1027: Hash recibido: mBI1Wmyzzk21Y, Salt: 01
Trabajador PID 1028: Hash recibido: mBI1Wmyzzk21Y, Salt: 01
Trabajador PID 1029: Hash recibido: mBI1Wmyzzk21Y, Salt: 01
br0285 #
```

```
#define _MINIX
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <unistd.h>
#include <sys/wait.h>
```

```

#include "generador.h"
#include "probar-clave.h"
#define OK 0

static void lanzar_trabajadores(const char alfabeto[], const int longitud, const int
num_trabajadores, const char* hash_salt) {
    int total = total_palabras();
    int i, inicio, fin;
    int palabras_por_trabajador = total / num_trabajadores;
    int resto = total % num_trabajadores;

    for (i = 0; i < num_trabajadores; i++) {
        pid_t pid = fork();
        if (pid == 0) {
            inicio = i * palabras_por_trabajador;
            fin = inicio + palabras_por_trabajador - 1;

            if (i == num_trabajadores - 1) {
                fin += resto;
            }
            printf("Trabajador PID %d: Hash recibido: %s, Salt: %s\n", getpid(),
hash_salt, alfabeto);
            exit(OK);
        } else if (pid < 0) {
            fprintf(stderr, "Error creacion proceso del trabajador %d\n", i);
        }
    }
    for (i = 0; i < num_trabajadores; i++) {
        wait(NULL);
    }
}

/**
 * Muestra un mensaje indicando cómo debe usarse el programa. Si se
 * modifican los argumentos del programa, hay que modificar esta función.
 */
static void uso(const char prog[]) {
    fprintf(stderr, "Uso: %s <alfabeto> <longitud> <num_trabajadores> <hash_salt>\n",
prog);
}

/**
 * Dada una cadena de caracteres, obtiene su representación numérica entera.
 * Tiene los siguientes parámetros:
 *  cadena: la versión textual del número.
 *  num: referencia al entero que albergará el valor numérico de la cadena.
 * Si cadena contiene una cadena numérica válida, devuelve OK.
 * En caso contrario, devuelve !OK y el valor referenciado por num es
 * indeterminado.
 */

static int str2int(const char cadena[], long *num) {
    char* endptr;

```

```

    errno = OK;
    *num = strtol(cadena, &endptr, 10);
    if (errno != OK || *endptr != '\0' || *num < 1) {
        return !OK;
    }
    return OK;
}

int main(const int argc, const char * argv[]) {
    long longitud, num_trabajadores;

    if (argc != 5) {
        uso(argv[0]);
        return !OK;
    }

    if (str2int(argv[2], &longitud) != OK) {
        fprintf(stderr, "La longitud no es válida\n");
        return !OK;
    }

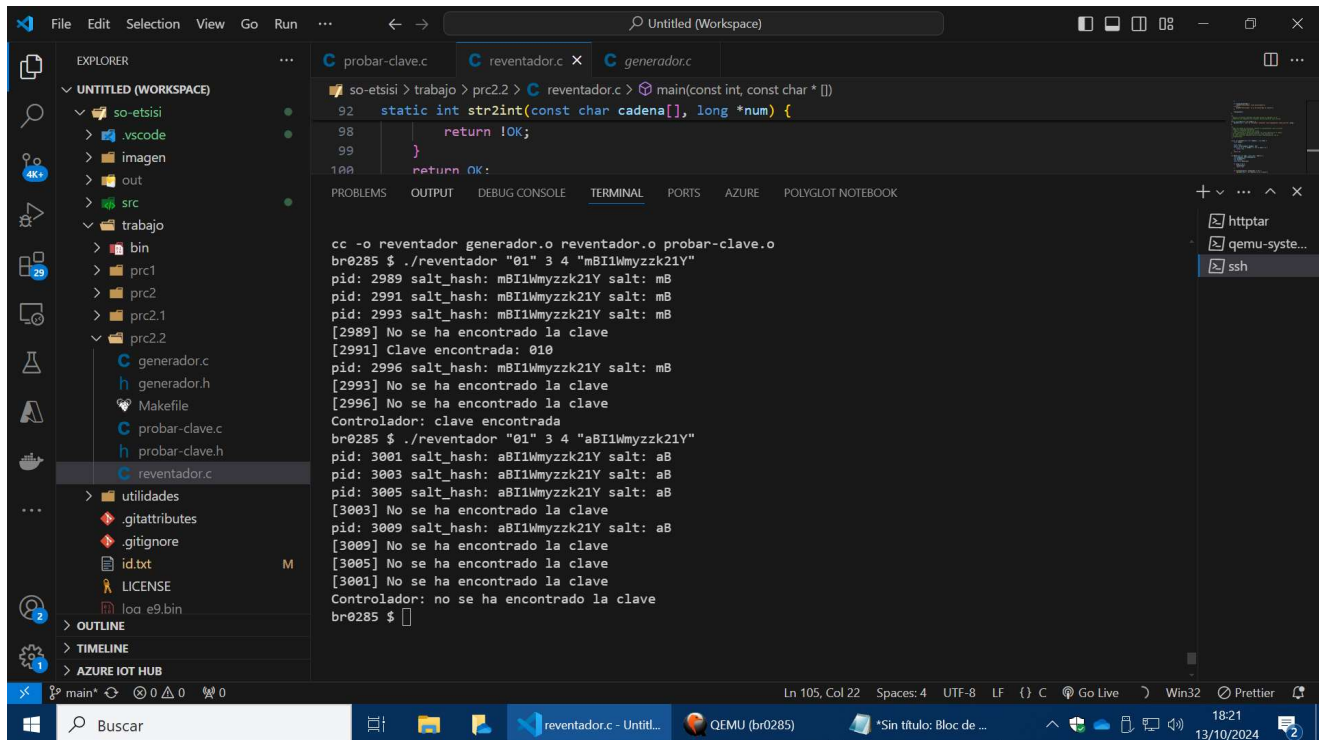
    if (str2int(argv[3], &num_trabajadores) != OK) {
        fprintf(stderr, "El número de trabajadores no es válido\n");
        return !OK;
    }

    if (configura_generador(argv[1], longitud) != OK) {
        fprintf(stderr, "El programa ha fallado\n");
        return !OK;
    }

    lanzar_trabajadores(argv[1], longitud, num_trabajadores, argv[4]);
    return OK;
}

```

3. Búsqueda clave



```
so-etsisi > trabajo > prc2.2 > C reventador.c > main(const int, const char * [])
92 static int str2int(const char cadena[], long *num) {
98     return !OK;
99 }
100 return OK;

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS AZURE POLYGLOT NOTEBOOK

cc -o reventador generator.o reventador.o probar-clave.o
br0285 $ ./reventador "01" 3 4 "mBI1Wmyzzk21Y"
pid: 2989 salt_hash: mBI1Wmyzzk21Y salt: mB
pid: 2991 salt_hash: mBI1Wmyzzk21Y salt: mB
pid: 2993 salt_hash: mBI1Wmyzzk21Y salt: mB
[2989] No se ha encontrado la clave
[2991] Clave encontrada: 010
pid: 2996 salt_hash: mBI1Wmyzzk21Y salt: mB
[2993] No se ha encontrado la clave
[2996] No se ha encontrado la clave
Controlador: clave encontrada
br0285 $ ./reventador "01" 3 4 "aBI1Wmyzzk21Y"
pid: 3001 salt_hash: aBI1Wmyzzk21Y salt: aB
pid: 3003 salt_hash: aBI1Wmyzzk21Y salt: aB
pid: 3005 salt_hash: aBI1Wmyzzk21Y salt: aB
[3003] No se ha encontrado la clave
pid: 3009 salt_hash: aBI1Wmyzzk21Y salt: aB
[3009] No se ha encontrado la clave
[3005] No se ha encontrado la clave
[3001] No se ha encontrado la clave
Controlador: no se ha encontrado la clave
br0285 $
```

```
#define _MINIX
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <unistd.h>
#include <sys/wait.h>
#include "generator.h"
#include "probar-clave.h"
#define OK 0

void obtener_palabra(int indice, char palabra[]);

static void lanzar_trabajadores(const char alfabeto[], const int longitud, const int
num_trabajadores, const char* hash_salt) {
    int total = total_palabras();
    int i, inicio, fin;
    int palabras_por_trabajador = total / num_trabajadores;
    int resto = total % num_trabajadores;
    int clave_encontrada = 0;
    int estado;
    pid_t pid;

    char *palabra = (char*)malloc((longitud + 1) * sizeof(char));
    if (palabra == NULL) {
        fprintf(stderr, "Error al asignar memoria\n");
        exit(1);
    }

    for (i = 0; i < num_trabajadores; i++) {
        pid = fork();
```

```

    if (pid == 0) {
        int j;
        inicio = i * palabras_por_trabajador;
        fin = inicio + palabras_por_trabajador - 1;
        if (i == num_trabajadores - 1) {
            fin += resto;
        }

        printf("pid: %d salt_hash: %s salt: %s\n", getpid(), hash_salt, alfabeto);

        for (j = inicio; j <= fin; j++) {
            genera_palabra(j, palabra);
            if (probar_clave(palabra, hash_salt, alfabeto) == 0) {
                printf("[%d] Clave encontrada: %s\n", getpid(), palabra);
                free(palabra);
                exit(OK);
            }
        }

        printf("[%d] No se ha encontrado la clave\n", getpid());
        free(palabra);
        exit(1);
    } else if (pid < 0) {
        fprintf(stderr, "Error creacion proceso del trabajador %d\n", i);
    }
}

for (i = 0; i < num_trabajadores; i++) {
    pid_t pid_hijo = wait(&estado);
    if (WIFEXITED(estado) && WEXITSTATUS(estado) == 0) {
        clave_encontrada = 1;
    }
}

if (clave_encontrada) {
    printf("Controlador: clave encontrada\n");
} else {
    printf("Controlador: no se ha encontrado la clave\n");
}

free(palabra);
}

/**
 * Muestra un mensaje indicando cómo debe usarse el programa. Si se
 * modifican los argumentos del programa, hay que modificar esta función.
 */
static void uso(const char prog[]) {
    fprintf(stderr, "Uso: %s <alfabeto> <longitud> <num_trabajadores> <hash_salt>\n",
prog);
}

/**

```

```

* Dada una cadena de caracteres, obtiene su representación numérica entera.
* Tiene los siguientes parámetros:
*  cadena: la versión textual del número.
*  num: referencia al entero que albergará el valor numérico de la cadena.
* Si cadena contiene una cadena numérica válida, devuelve OK.
* En caso contrario, devuelve !OK y el valor referenciado por num es
* indeterminado.
*/

```

```

static int str2int(const char cadena[], long *num) {
    char* endptr;

    errno = OK;
    *num = strtol(cadena, &endptr, 10);
    if (errno != OK || *endptr != '\0' || *num < 1) {
        return !OK;
    }
    return OK;
}

```

```

int main(const int argc, const char * argv[]) {
    long longitud, num_trabajadores;
    char alfabeto[3];
    char salt[3];
    const char* hash_salt;

    if (argc != 5) {
        uso(argv[0]);
        return !OK;
    }

    if (str2int(argv[2], &longitud) != OK) {
        fprintf(stderr, "La longitud no es válida\n");
        return !OK;
    }

    if (str2int(argv[3], &num_trabajadores) != OK) {
        fprintf(stderr, "El número de trabajadores no es válido\n");
        return !OK;
    }

    alfabeto[0] = argv[1][0];
    alfabeto[1] = argv[1][1];
    alfabeto[2] = '\0';

    salt[0] = argv[4][0];
    salt[1] = argv[4][1];
    salt[2] = '\0';

    hash_salt = argv[4];

    if (configura_generador(alfabeto, longitud) != OK) {
        fprintf(stderr, "El programa ha fallado\n");
        return !OK;
    }
}

```



```

}

lanzar_trabajadores(salt, longitud, num_trabajadores, hash_salt);
return OK;
}

```

4. Señales

```

cc -o reventador generador.o reventador.o probar-clave.o
br0285 $ ./reventador "01abcdefg" 3 4 "mBI1Wmyzzk21Y"
pid: 438 salt_hash: mBI1Wmyzzk21Y
pid: 440 salt_hash: mBI1Wmyzzk21Y
pid: 443 salt_hash: mBI1Wmyzzk21Y
[443] No se ha encontrado la clave
pid: 446 salt_hash: mBI1Wmyzzk21Y
El trabajador 443 terminó sin encontrar la clave.
[440] Clave encontrada: 010
El trabajador 440 ha encontrado la clave.
[446] Recibí una señal tipo SIGTERM, abortando búsqueda.
El trabajador 446 ha finalizado la búsqueda.
[438] Recibí una señal tipo SIGTERM, abortando búsqueda.
El trabajador 438 ha finalizado la búsqueda.
Clave encontrada
br0285 $ ./reventador "01abcdefg" 3 4 "aBI1Wmyzzk21Y"
pid: 449 salt_hash: aBI1Wmyzzk21Y
pid: 451 salt_hash: aBI1Wmyzzk21Y
pid: 453 salt_hash: aBI1Wmyzzk21Y
[451] No se ha encontrado la clave
pid: 457 salt_hash: aBI1Wmyzzk21Y
[457] No se ha encontrado la clave
El trabajador 451 terminó sin encontrar la clave.
El trabajador 457 terminó sin encontrar la clave.
[453] No se ha encontrado la clave
El trabajador 453 terminó sin encontrar la clave.
[449] No se ha encontrado la clave
El trabajador 449 terminó sin encontrar la clave.
No se ha encontrado la clave
br0285 $

```

```

#define _MINIX
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <unistd.h>
#include <sys/wait.h>
#include <signal.h>
#include <sys/types.h>
#include "generador.h"
#include "probar-clave.h"
#define OK 0
#define ERROR 1

static pid_t *trabajadores_activos;
static int num_trabajadores_activos;

void manejador_sigterm(int sig) {
    printf("[%d] Recibí una señal tipo SIGTERM, abortando búsqueda.\n", getpid());
    exit(2);
}

void eliminar_trabajador_activo(pid_t pid) {

```

```

    int i;
    for (i = 0; i < num_trabajadores_activos; i++) {
        if (trabajadores_activos[i] == pid) {
            trabajadores_activos[i] = trabajadores_activos[num_trabajadores_activos - 1];
            num_trabajadores_activos--;
            break;
        }
    }
}

void terminar_trabajadores(void) {
    int i;
    for (i = 0; i < num_trabajadores_activos; i++) {
        kill(trabajadores_activos[i], SIGTERM);
    }
}

static void lanzar_trabajadores(const char alfabeto[], int longitud, int num_trabajadores,
const char* hash_salt) {
    int total, palabras_por_trabajador, resto, clave_encontrada, estado;
    pid_t pid;
    char *palabra;
    int i, j, inicio, fin;

    total = (int) total_palabras();
    palabras_por_trabajador = total / num_trabajadores;
    resto = total % num_trabajadores;
    clave_encontrada = 0;

    palabra = (char*)malloc((longitud + 1) * sizeof(char));
    if (palabra == NULL) {
        fprintf(stderr, "Error al asignar memoria\n");
        exit(ERROR);
    }

    trabajadores_activos = (pid_t*) malloc(num_trabajadores * sizeof(pid_t));
    num_trabajadores_activos = 0;

    for (i = 0; i < num_trabajadores; i++) {
        pid = fork();
        if (pid == 0) {
            signal(SIGTERM, manejador_sigterm);

            inicio = i * palabras_por_trabajador;
            fin = inicio + palabras_por_trabajador - 1;
            if (i == num_trabajadores - 1) {
                fin += resto;
            }

            printf("pid: %d salt_hash: %s\n", getpid(), hash_salt);

            for (j = inicio; j <= fin; j++) {
                genera_palabra(j, palabra);
                if (probar_clave(palabra, hash_salt, alfabeto) == OK) {

```

```

        printf("[%d] Clave encontrada: %s\n", getpid(), palabra);
        free(palabra);
        exit(OK);
    }
}

printf("[%d] No se ha encontrado la clave\n", getpid());
free(palabra);
exit(1);
} else if (pid > 0) {
    trabajadores_activos[num_trabajadores_activos++] = pid;
} else {
    fprintf(stderr, "Error en la creación del proceso del trabajador %d\n", i);
}
}

for (i = 0; i < num_trabajadores; i++) {
    pid_t pid_hijo = wait(&estado);
    eliminar_trabajador_activo(pid_hijo);

    if (WIFEXITED(estados)) {
        int codigo_salida = WEXITSTATUS(estados);

        if (codigo_salida == OK) {
            clave_encontrada = 1;
            printf("El trabajador %d ha encontrado la clave.\n", pid_hijo);
            terminar_trabajadores();
        } else if (codigo_salida == 1) {
            printf("El trabajador %d terminó sin encontrar la clave.\n", pid_hijo);
        } else if (codigo_salida == 2) {
            printf("El trabajador %d ha finalizado la búsqueda.\n", pid_hijo);
        }
    }
}

if (clave_encontrada) {
    printf("Clave encontrada\n");
} else {
    printf("No se ha encontrado la clave\n");
}

free(palabra);
free(trabajadores_activos);
}

static void uso(const char prog[]) {
    fprintf(stderr, "Uso: %s <alfabeto> <longitud> <num_trabajadores> <hash_salt>\n",
prog);
}

static int str2int(const char cadena[], long *num) {
    char* endptr;

    errno = OK;

```

```

    *num = strtol(cadena, &endptr, 10);
    if (errno != OK || *endptr != '\0' || *num < 1) {
        return !OK;
    }
    return OK;
}

int main(const int argc, const char * argv[]) {
    long longitud;
    int num_trabajadores;
    char alfabeto[3];
    char salt[3];
    const char* hash_salt;

    if (argc != 5) {
        uso(argv[0]);
        return !OK;
    }

    if (str2int(argv[2], &longitud) != OK) {
        fprintf(stderr, "La longitud no es válida\n");
        return !OK;
    }

    if (str2int(argv[3], (long*)&num_trabajadores) != OK) {
        fprintf(stderr, "El número de trabajadores no es válido\n");
        return !OK;
    }

    alfabeto[0] = argv[1][0];
    alfabeto[1] = argv[1][1];
    alfabeto[2] = '\0';

    salt[0] = argv[4][0];
    salt[1] = argv[4][1];
    salt[2] = '\0';

    hash_salt = argv[4];

    if (configura_generador(alfabeto, (int)longitud) != OK) {
        fprintf(stderr, "Error al configurar el generador\n");
        return !OK;
    }

    lanzar_trabajadores(salt, (int)longitud, num_trabajadores, hash_salt);
    return OK;
}

```